Manual técnico — Neko Sensei

Oct 30, 2026 · Luis Enrique Valey Osorio (9490-21-16222)

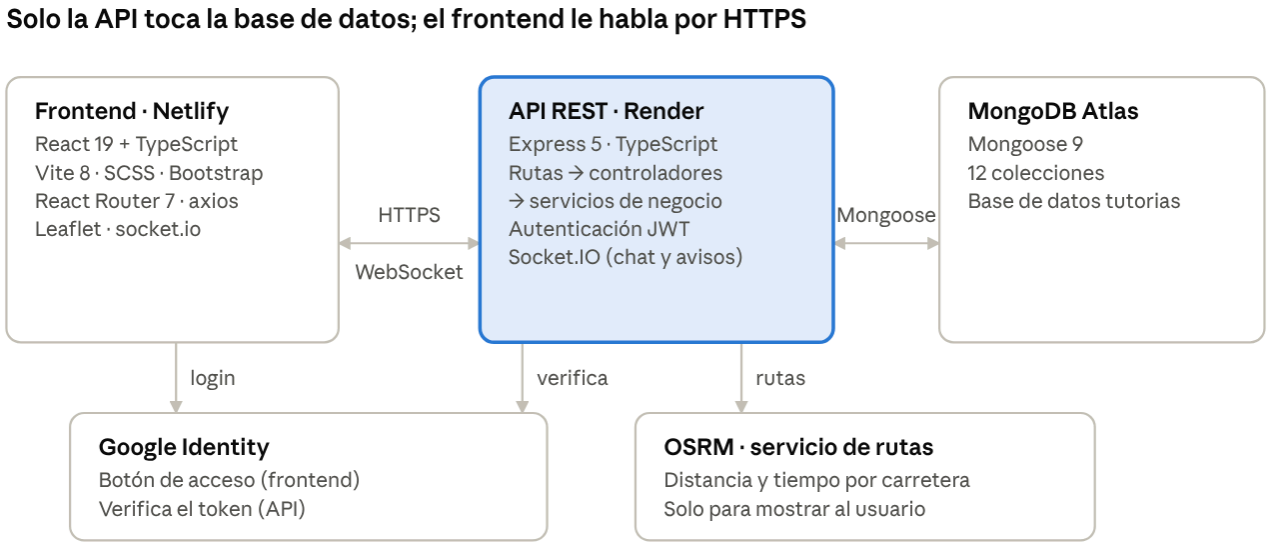
1. Introducción y alcance

Neko Sensei es una plataforma web que conecta estudiantes con tutores: el estudiante busca un curso, lo solicita, paga por bloques de clases y las toma en línea o a domicilio; el tutor publica ofertas y atiende a sus alumnos. Este manual explica cómo está construida y cómo operarla, y está dirigido a quien deba instalarla, mantenerla o ampliarla.

- **Roles:** visitante, estudiante, tutor y administrador.
- **Repositorio:** www.github.com/lvaley/NekoSensei-V08.

2. Arquitectura general

El navegador carga el frontend desde Netlify y consume la API en Render por HTTPS y WebSocket; solo la API accede a MongoDB Atlas.



arquitectura · 3 capas y 2 servicios externos

Google Identity se usa para iniciar sesión y OSRM solo para mostrar la distancia y el tiempo por carretera de las clases a domicilio. El cobro del traslado se calcula en línea recta, sin depender de OSRM.

3. Tecnologías y versiones

Todo el proyecto está escrito en TypeScript y corre sobre Node 22. Las versiones son las declaradas en los `package.json` (rangos `^`).

Capa	Tecnología	Versión	Para qué se usa
Frontend	React	19	Interfaz y componentes
Frontend	Vite	8	Servidor de desarrollo y compilación
Frontend	React Router	7	Rutas y rutas protegidas por rol
Frontend	Bootstrap / React-Bootstrap	5.3 / 2.10	Componentes y rejilla
Frontend	SCSS (sass)	1.105	Estilos propios en parciales
Frontend	Leaflet / react-leaflet	1.9 / 5	Mapas y selector de ubicación
Frontend	axios	1.x	Cliente HTTP con el JWT en cada petición
Frontend	@react-oauth/google	0.13	Botón de inicio de sesión con Google
Frontend	socket.io-client	4.8	Chat y avisos en tiempo real
Frontend	ESLint	10	Análisis estático (reglas de react-hooks)
Backend	Express	5	API REST
Backend	Mongoose	9	Modelos y acceso a MongoDB
Backend	socket.io	4.8	Chat y avisos en tiempo real
Backend	jsonwebtoken	9	Sesión por JWT
Backend	google-auth-library	11	Verifica el token de Google
Backend	helmet, cors, morgan	8 / 2 / 1	Cabeceras de seguridad, CORS y bitácora
Backend	TypeScript	7	Compilación a `dist/` con `tsc`
Datos	MongoDB Atlas	—	Base de datos en la nube
Servicios	Netlify / Render	—	Hosting del frontend / de la API

3.1 Uso de TypeScript

Todo el código, de frontend y backend, está en TypeScript con tipado estricto: los errores de tipos se detectan al compilar y no cuando el usuario ya está usando la plataforma.

- **Backend (`backend/tsconfig.json`).** Compila con `tsc` de `src/` a `dist/` (módulos CommonJS, ES2022). Usa `strict`, `noUncheckedIndexedAccess` (todo acceso a arreglos o índices puede ser `undefined` y se debe comprobar), `noImplicitReturns` y `noUnusedLocals/Parameters`.
- **Frontend (`tsconfig.app.json`).** Vite compila sin emitir archivos (`noEmit`); `npm run build` ejecuta primero la revisión de tipos. Usa `jsx: react-jsx`, `verbatimModuleSyntax` (los tipos se importan con `import type`) y `erasableSyntaxOnly` (sin `enum` ni sintaxis que requiera transformación).
- **Tipos compartidos.** En el frontend, `src/types/index.ts` define las formas de los datos que devuelve la API (`AuthUser`, `TutorProfile`, `Appeal`, `Suspension`, analítica, pagos, etc.) y uniones literales como `UserRole = 'student' | 'tutor' | 'admin'` o `TutorStatus`. En el backend, cada colección tiene su interfaz en `models/`.
- **Convenciones.** Componentes y páginas son funciones tipadas (`.tsx`); las respuestas de la API se tipan en el cliente axios; los estados posibles (rol, estado de inscripción, estado del tutor) se modelan como uniones de cadenas, no como textos libres.

Comandos: `npm run typecheck` (backend) revisa tipos sin compilar y `npm run build` en cada proyecto los revisa antes de generar la salida.

3.2 Estilos con Sass/SCSS

Los estilos propios están en `src/styles/` y se compilan con Sass. Bootstrap 5 aporta la rejilla y los componentes base, y se personaliza con variables de Sass antes de importarlo, sin tocar sus archivos.

Archivo	Función
`main.scss`	Punto de entrada: usa (`@use`) variables, mixins y parciales; sobrescribe variables de Bootstrap (`$primary`, `$body-bg`, tipografía, acordeón) y al final importa `bootstrap/scss/bootstrap`
`abstracts/_variables.scss`	Paleta (tinta, papel, bermellón, dorado), blancos translúcidos para fondos oscuros, tipografía, pesos y puntos de quiebre

Archivo	Función
`abstracts/_mixins.scss`	Utilidades reutilizables: `tablet-up` y `desktop-up` (media queries), `card-surface`, `pill-button`, `label-text`, `gold-hover`, `hero-banner`
`components/_navbar.scss`, `_footer.scss`, `_home.scss`, `_pages.scss`, `_skeleton.scss`	Estilos por área de la interfaz

Cómo se usa.

- Los colores y tamaños se toman siempre de variables (`$vermilion`, `$gold`, `$line`); no se escriben valores sueltos en los parciales.
- Se usa `@use` en lugar de `@import` (módulos de Sass), y funciones modernas como `color.change` para las variantes translúcidas.
- El diseño es mobile-first: los estilos base son para móvil y los mixins `tablet-up` / `desktop-up` agregan las reglas de pantallas mayores.
- Para una pantalla nueva se agrega una clase en el parcial correspondiente (`_pages.scss` para páginas) reutilizando los mixins, y se usan las clases de Bootstrap para la maquetación.

4. Estructura del repositorio

La aplicación web (frontend) vive en la raíz del repositorio y el servidor (API) en la carpeta `backend`; los archivos de despliegue también están en la raíz.

Ruta	Contenido
`render.yaml`	Blueprint de Render para la API
`README.md`	Resumen del proyecto, hoja de ruta y notas de cada versión
`backend/src/server.ts`	Arranque: conecta a MongoDB, crea el catálogo inicial y levanta HTTP + sockets
`backend/src/app.ts`	Express: helmet, CORS, JSON, bitácora y rutas bajo `/api`
`backend/src/routes/index.ts`	Todas las rutas y qué rol puede usarlas
`backend/src/controllers/`	Un archivo por área (inscripciones, pagos, sesiones, denuncias, reportería…)

Ruta	Contenido
`backend/src/services/`	Lógica de negocio reutilizable: precios, horarios semanales, traslado, notificaciones, sockets
`backend/src/models/`	Esquemas de Mongoose (12 colecciones)
`backend/src/middlewares/auth.ts`	`requireAuth`, `requireAuthAllowSuspended` y `requireRole`
`src/pages/`	Una página por ruta
`src/components/`	Componentes por área (layout, ui, charts, enrollments, reports…)
`src/routes/AppRouter.tsx`	Mapa de rutas públicas, protegidas y de administración
`src/context/`	Sesión (`AuthProvider`) y chat (`ChatProvider`)
`src/services/api.ts`	Cliente axios: agrega el JWT y avisa cuando la cuenta está suspendida
`src/config/site.ts` y `content/`	Datos editables: marca, contacto, preguntas frecuentes y textos legales
`src/styles/`	SCSS: variables, mixins y parciales por área
`netlify.toml`	Compilación y redirección SPA en Netlify

5. Instalación y ejecución local

Se necesita Node 22 y una base MongoDB (un clúster gratuito de Atlas sirve). La API corre en el puerto 4000 y el frontend en el 5173.

1. Clonar el repositorio y entrar a la carpeta `backend`.
2. Instalar las dependencias y crear el archivo de entorno a partir del ejemplo:

```
cd backend
npm install
cp .env.example .env
```

3. Completar `backend/.env` con la cadena de conexión (`MONGODB_URI`), un secreto para las sesiones (`JWT_SECRET`), el ID de cliente de Google (`GOOGLE_CLIENT_ID`) y el correo del administrador (`ADMIN_EMAILS`); ver la sección 6. Luego iniciar la API:

```
npm run dev
```

4. En otra terminal, volver a la raíz del repositorio (donde está el frontend) y prepararlo con su propio `.env` (`VITE_API_URL` y `VITE_GOOGLE_CLIENT_ID`):

```
cd ..
npm install
npm run dev
```

5. Abrir http://localhost:5173. Al iniciar la API por primera vez se crea el catálogo de categorías inicial, y el correo indicado en `ADMIN_EMAILS` recibe el rol de administrador al iniciar sesión con Google.

Scripts disponibles:

Proyecto	Comando	Qué hace
backend	`npm run dev`	API con recarga automática (tsx watch)
backend	`npm run build`	Compila a `dist/` con tsc
backend	`npm start`	Ejecuta `dist/server.js` (producción)
backend	`npm run typecheck`	Revisa tipos sin compilar
frontend	`npm run dev`	Servidor de desarrollo de Vite
frontend	`npm run build`	Revisa tipos y genera `dist/`
frontend	`npm run lint`	ESLint
frontend	`npm run preview`	Sirve el `dist/` compilado

6. Variables de entorno

La API no arranca sin `MONGODB_URI` , y en producción exige un `JWT_SECRET` propio. Las variables del frontend empiezan con `VITE_` y se incorporan al compilar: si cambian, hay que volver a desplegar.

Proyecto	Variable	Obligatoria	Valor por defecto / ejemplo
backend	`MONGODB_URI`	Sí	`mongodb+srv://<usuario>:<password>@<cluster>.mongodb.net/tutorias`
backend	`JWT_SECRET`	Sí en producción	Cadena larga y aleatoria
backend	`JWT_EXPIRES_IN`	No	`1h`

Proyecto	Variable	Obligatoria	Valor por defecto / ejemplo
backend	`GOOGLE_CLIENT_ID`	Sí para iniciar sesión	Client ID de tipo "Aplicación web"
backend	`ADMIN_EMAILS`	Sí (al menos uno)	Correos separados por coma que reciben el rol administrador
backend	`CLIENT_URL`	Sí en producción	URL del frontend; admite varias separadas por coma, sin barra final
backend	`PORT`	No	`4000` (Render lo asigna solo)
backend	`NODE_ENV`	No	`development` o `production`
backend	`OSRM_URL`	No	Servidor público de OSRM
backend	`COMMISSION_RATE`	No	`0.10` (10 % de comisión)
frontend	`VITE_API_URL`	Sí	`http://localhost:4000/api` en local; `https://<tu-api>.onrender.com/api` en producción
frontend	`VITE_GOOGLE_CLIENT_ID`	Sí	El mismo Client ID de Google (es público)

El archivo `.env` nunca debe subirse al repositorio (ya está en. Los datos de contacto que ven los usuarios se editan en `src/config/site.ts` y `src/content/contact.ts`.

7. Modelo de datos

La base tiene 12 colecciones. Las siete centrales se relacionan así; el resto se resume en la tabla de abajo.

Todo gira en torno a la inscripción: clases, pagos y reseña cuelgan de ella



modelo de datos · 7 colecciones centrales

Colección	Qué guarda
`Report`	Denuncias entre estudiante y tutor: motivo, estado y quién la resolvió
`Notification`	Avisos de la campanita, por usuario (inscripciones, pagos, clases, moderación, administración)
`Conversation`	Chat entre un estudiante y un tutor
`Message`	Mensajes de una conversación
`ContactMessage`	Mensajes del formulario de contacto (no se ligan a un usuario)

Decisiones de diseño a tener presentes:

- La inscripción copia los datos del curso al momento de solicitarlo (título, tarifa, plan). Si el tutor edita o borra la oferta, el historial no cambia.
- Del pago solo se guarda la marca y los últimos 4 dígitos de la tarjeta; nunca el número completo ni el CVV.
- La reseña es única por inscripción y no se edita.
- El PDF de la hoja de vida del tutor se guarda en el usuario y no se incluye en las consultas normales.

8. API REST

Toda la API vive bajo `/api`; las rutas están en `backend/src/routes/index.ts` junto con el rol que puede usarlas. Las respuestas son JSON y los errores llevan un campo `message` en español. Las peticiones autenticadas envían `Authorization: Bearer <JWT>`.

Grupo	Endpoints principales	Quién
Estado	`GET /health`	Público
Catálogo público	`GET /public/categories`, `/public/stats`, `/tutors`, `/tutors/:id`, `/tutors/:id/reviews`, `/offers`	Público
Contacto	`POST /contact`	Público
Sesión	`POST /auth/google`, `GET /auth/me`	Público / con sesión
Perfil	`PUT /profile`	Con sesión
Ofertas del tutor	`GET/POST /my/offers`, `PUT/DELETE /my/offers/:id`	Tutor aprobado
Inscripciones	`POST/GET /enrollments`, `PATCH /enrollments/:id/decision`, `/cancel`, `/finish`	Estudiante y tutor
Pagos	`GET /offers/:id/plan-slots`, `POST /enrollments/:id/pay`, `/renew`, `GET /payments`	Estudiante y tutor
Clases	`GET /sessions`, `POST /enrollments/:id/sessions`, `PATCH /sessions/:id/cancel`, `/complete`, `GET /sessions/:id/detail`	Estudiante y tutor
Reagendar	`POST /sessions/:id/reschedule`, `/propose`, `/decline`, `/respond`	Estudiante y tutor
Traslado	`GET /offers/:id/travel`, `GET /enrollments/:id/travel`	Estudiante
Reseñas	`POST /enrollments/:id/review`	Estudiante
Chat	`GET/POST /conversations`, `GET/POST /conversations/:id/messages`, `POST /conversations/:id/read`	Estudiante y tutor
Notificaciones	`GET /notifications`, `PATCH /notifications/read-all`, `/:id/read`	Con sesión

Grupo	Endpoints principales	Quién
Denuncias	`POST /reports`	Estudiante y tutor
Suspensión	`POST /suspension/appeal`	Cuenta suspendida
Ganancias	`GET /tutor/earnings`, `POST /tutor/reactivation`	Tutor
Administración	`/users`, `/admin/overview`, `/admin/analytics`, `/admin/tutors`, `/admin/reports`, `/admin/suspensions`, `/admin/contact-messages`, `/categories` (crear, editar, borrar)	Administrador

El listado completo y exacto está en el archivo de rutas; ante una duda, ese archivo manda sobre esta tabla.

9. Autenticación, roles y moderación

No hay contraseñas: el acceso es solo con Google, y la API entrega su propio JWT para identificar la sesión.

1. El frontend muestra el botón de Google (`VITE_GOOGLE_CLIENT_ID`) y recibe una credencial.
2. Envía esa credencial a `POST /auth/google` ; la API la verifica con `google-auth-library` y crea o actualiza el usuario.
3. La API responde con un JWT (dura `JWT_EXPIRES_IN` , 1 hora por defecto) que axios agrega a cada petición.
4. En cada petición protegida, `requireAuth` valida el JWT, carga al usuario y rechaza las cuentas suspendidas con `403` y el código `account_suspended` .

Roles. Estudiante y tutor se eligen al registrarse. Administrador solo lo obtiene quien tenga su correo en `ADMIN_EMAILS` . `requireRole('admin')` y similares protegen las rutas; en el frontend, `ProtectedRoute` hace lo mismo con las páginas.

Perfil de tutor. Su estado controla qué puede hacer:

Estado	Significado	Efecto
`pending`	Perfil enviado, en revisión	No publica ofertas ni aparece en el catálogo

Estado	Significado	Efecto
`approved`	Aprobado por el administrador	Publica ofertas, aparece en el catálogo y recibe solicitudes
`rejected`	Rechazado con motivo	Puede corregir y reenviar
`inactive`	Inactivado por el administrador, con motivo	Sale del catálogo y no recibe solicitudes ni renovaciones; sus alumnos actuales terminan sus cursos. Puede pedir reactivación

Moderación.

- Estudiantes y tutores solo pueden denunciarse entre sí si comparten una inscripción, y solo hay una denuncia abierta por pareja.
- El administrador revisa las denuncias y puede suspender la cuenta; al suspender se resuelven todas las denuncias abiertas contra ella.
- Una cuenta suspendida puede iniciar sesión, pero solo ve la pantalla de suspensión (motivo, normas de convivencia y formulario de apelación). El administrador acepta la apelación (se reactiva la cuenta) o la rechaza con una respuesta.
- Los tutores suspendidos desaparecen del catálogo y el chat los rechaza.

10. Reglas de negocio clave

Estas son las reglas que más impactan el comportamiento de la plataforma. Cualquier cambio debe hacerse en los controladores y servicios del backend, nunca solo en el frontend.

Ciclo de una inscripción.

Estado	Cuándo ocurre	Siguiente paso
`pending`	El estudiante solicita un curso y elige sus horarios semanales fijos	El tutor acepta (→ `awaiting_payment`) o rechaza (→ `rejected`)
`awaiting_payment`	El tutor aceptó	El estudiante tiene 48 horas para pagar; si no, pasa a `expired`
`active`	Pago realizado	Las clases se van impartiendo; al terminar el bloque se puede renovar

Estado	Cuándo ocurre	Siguiente paso
`finished`	Se cumplieron todas las clases	Estado final; el estudiante puede reseñar
`cancelled`	Cancelación por estudiante o tutor	Estado final; si cancela el tutor se aplica reembolso
`rejected` / `expired`	Rechazo del tutor o vencimiento del plazo de pago	Estado final

Pagos.

- El pago es simulado: solo se guardan la marca y los últimos 4 dígitos de la tarjeta, nunca el número completo ni el CVV.
- Cada pago cubre un bloque de clases; renovar crea un nuevo pago sobre la misma inscripción. No se puede renovar si el tutor ya no está aprobado.
- Las reglas de tarjetas de prueba (válidas, rechazadas, vencidas) viven en el servicio de pagos del backend.

Reembolsos. Solo se generan cuando el **tutor** cancela una inscripción activa. El monto es proporcional a las clases no impartidas del último bloque pagado. Si la inscripción termina normalmente no hay reembolso. El pago guarda `refundedAmount` y `refundedAt`, y el estudiante recibe una notificación.

Costo de traslado. En cursos presenciales a domicilio se suma: distancia en línea recta (fórmula de Haversine) × tarifa por km del tutor × número de sesiones. El mapa usa OSRM solo para mostrar la ruta; no interviene en el cobro.

Comisión. La plataforma retiene `COMMISSION_RATE` (10 % por defecto) del ingreso bruto menos los reembolsos. Ver sección 12.

11. Tiempo real y notificaciones

Chat (socket.io). El servidor de socket.io comparte el servidor HTTP de Express. El cliente se conecta con su JWT; al conectar, el servidor lo valida y rechaza a las cuentas suspendidas. Cada conversación es una sala y los mensajes se guardan en MongoDB (`Conversation` y `Message`) antes de emitirse, por lo que el historial sobrevive a recargas y desconexiones.

Notificaciones. Cada evento relevante (solicitud nueva, inscripción aceptada, pago, reembolso, denuncia, suspensión, aprobación o inactivación de tutor, solicitud de reactivación) crea un documento `Notification` para el usuario destinatario. Para avisar a

todos los administradores se usa el helper `notifyAdmins`. El frontend las consulta por la campana de la barra superior y las marca como leídas.

Para agregar un tipo nuevo: añadirlo a la lista de tipos en `models/Notification.ts`, crearlo desde el controlador que corresponda y, si debe mostrarse con un ícono o enlace propio, ajustar el componente de la campana en el frontend.

CORS y sockets. socket.io usa los mismos orígenes permitidos que la API (`CLIENT_URL`); si el chat falla en producción y la API responde bien, revisar primero esa variable.

12. Reportería y comisión

Endpoint	Rol	Contenido
`GET /admin/analytics`	Admin	Ingresos y comisión por mes, ranking de tutores y cursos, demanda por categoría, tutorías impartidas vs. canceladas, peor calificados y reembolsos
`GET /tutor/earnings`	Tutor	Ganancias netas, comisión descontada, reembolsos y detalle por mes
`GET /admin/overview`	Admin	Pendientes y cifras del panel principal

Cálculo. Ingreso neto de un tutor = pagos − reembolsos − comisión. La comisión es `COMMISSION_RATE` × (bruto − reembolsos).

Criterios de fecha.

- Los meses se miden en hora de Guatemala (UTC−6), no en UTC, para que un pago de la noche no caiga en el mes siguiente.
- Un reembolso se cuenta en el mes del pago original, no en el de la cancelación; así los totales mensuales no se alteran retroactivamente de forma confusa.

Gráficas. Son SVG propios (`components/charts/BarChart.tsx` y `HBars.tsx`), sin librerías externas. La reportería es solo en pantalla; no hay exportación.

13. Pruebas y calidad

Antes de cada entrega se ejecutan estas verificaciones; todas deben terminar sin errores.

Capa	Comando	Qué valida
Backend	`npm run build` (en `backend/`)	Compilación TypeScript estricta (`noUncheckedIndexedAccess`)

Capa	Comando	Qué valida
Frontend	`npm run build` (en la raíz)	Tipos y empaquetado de Vite
Frontend	`npm run lint`	ESLint, incluidas las reglas de hooks de React
API	Scripts de prueba contra la API real	Flujos de negocio: inscripción, pago, reembolso, moderación, tutor inactivo, reportería
Navegador	Pruebas de extremo a extremo con Playwright	Recorridos completos por rol (estudiante, tutor, administrador)

Base de datos de pruebas. Las pruebas de API corren contra una instancia local compatible con MongoDB (FerretDB sobre SQLite), de modo que nunca tocan los datos de Atlas.

Prácticas recomendadas.

- Probar con los tres roles después de cualquier cambio en autenticación o rutas protegidas.
- Revisar que una cuenta suspendida y un tutor inactivo vean lo esperado.
- Si se agregan variables `VITE_*`, volver a desplegar el frontend: se incrustan al compilar.

14. Despliegue en Netlify, Render y Atlas

Orden recomendado: base de datos, API, frontend y, al final, ajustar orígenes.

1. **MongoDB Atlas.** Crear el clúster, un usuario de base de datos y, en *Network Access*, permitir `0.0.0.0/0` (Render usa IPs variables). Copiar la cadena de conexión a `MONGODB_URI`.
2. **Render (API).** Crear un *Web Service* desde el repositorio. El archivo `render.yaml` de la raíz ya define: directorio raíz `backend`, build `npm install --include=dev && npm run build`, inicio `npm start` y health check `/api/health`. Cargar las variables de la sección 6 (`MONGODB_URI`, `JWT_SECRET`, `GOOGLE_CLIENT_ID`, `ADMIN_EMAILS`, `CLIENT_URL`, `COMMISSION_RATE`).
3. **Netlify (frontend).** Importar el repositorio y dejar *Base directory* en blanco, porque el frontend está en la raíz. `netlify.toml` fija Node 22, el comando de build, el directorio `dist` y la redirección de SPA. Definir `VITE_API_URL` (URL de Render con `/api`) y `VITE_GOOGLE_CLIENT_ID`.

4. **CORS.** Poner la URL de Netlify en `CLIENT_URL` de Render (varias, separadas por comas; se ignora la barra final).
5. **Google OAuth.** En Google Cloud Console, agregar la URL de Netlify (y `http://localhost:5173` para desarrollo) a los *orígenes JavaScript autorizados* del ID de cliente.
6. **Verificar.** Abrir `/api/health` en Render, iniciar sesión con Google y revisar la página de estado del sistema (administradores).

Notas. Un cambio en `VITE_*` exige un nuevo despliegue en Netlify.

15. Mantenimiento y solución de problemas

Síntoma	Causa probable	Solución
El frontend carga pero todas las llamadas fallan por CORS	`CLIENT_URL` no incluye el origen de Netlify	Corregir la variable en Render y reiniciar el servicio
Primera petición muy lenta	Render gratuito despertó de su suspensión	Esperar; opcionalmente usar un servicio de ping al `/api/health`
El botón de Google falla o dice "origen no autorizado"	La URL no está en los orígenes autorizados del ID de cliente	Agregarla en Google Cloud Console
Cambié `VITE_API_URL` y no surte efecto	Las variables `VITE_*` se fijan al compilar	Redesplegar en Netlify
Un usuario no ve el panel de administrador	Su correo no está en `ADMIN_EMAILS`	Agregarlo, reiniciar la API y volver a iniciar sesión
Error de conexión a MongoDB	IP no permitida o credenciales incorrectas en Atlas	Revisar *Network Access* y `MONGODB_URI`
Recarga de una ruta del frontend da 404	Falta la redirección de SPA	Verificar que `netlify.toml` esté en la raíz del repositorio
Un ícono de categoría no aparece	Lista del backend y del frontend desincronizadas	Mantener iguales `CATEGORY_ICONS` (backend) y `CATEGORY_ICON_OPTIONS` (frontend)

Tareas periódicas.

- Actualizar dependencias con `npm outdated` y volver a correr la sección 13.

- Rotar `JWT_SECRET` si se sospecha filtración (cierra todas las sesiones).
- Revisar denuncias y tutores pendientes desde el panel de administrador.